

pg_txid_xmin_age Saturation Alert

Symptoms – how pg_txid_xmin_age may be associated with various issues

First of all, understand if there are ongoing issues that may be associated to the ongoing pg_txid_xmin_age spike (an example for one of past incidents):

- SubtransControlLock spikes on standby nodes – check Thanos, using this query:

```
sum by (fqdn) (avg_over_time(pg_stat_activity_marginalia_sampler_active_count{env="gprc"}
```

If you see a spike, this is a serious issue – most likely there are 5xx errors happening already, affecting users.

- Locking issues – a grown number of slow queries (>1s) and timed-out (reached 15s) queries, affecting users (if we deal with a long-running transaction holding many locks, blocking other sessions).
- Autovacuum cannot delete tuples that became dead during the ongoing pg_txid_xmin_age spike – can be checked in PostgreSQL log (growing numbers of “XXXX are dead but not yet removable” reported when autoVACUUM finishes processing a table). This doesn’t hurt immediately, 10-20 minutes is not a big deal, but it contributes to the bloat growth. A spike lasting an hour would definitely cause serious bloat growth for some tables.

What to do about it

Depending on symptoms you observe, you may decide to choose:

- immediate intervention (see below) if symptoms are severe, or
- “watch and wait” tactics, being ready to act (if there are no serious issues).

Diagnostics: root cause of pg_txid_xmin_age spike

The spike of pg_txid_xmin_age can be caused by one of the following 3 reasons:

1. AutoANALYZE on one of large tables
 - if it finished, it can be checked in PostgreSQL logs, searching for “automatic analyze” and paying attention to the last value (elapsed: XXX s)
 - if it’s still happening, logs won’t show it – it is reported only in the end. But we can look at pg_stat_activity – search for query ~ '^autovacuum and pay attention to those entries that are running ANALYZE (note that VACUUM entries are not harmful – they do not hold snapshot, so they do not cause pg_txid_xmin_age spikes), and there is backend_xmin value. Example snippet to use:

```

select
    clock_timestamp() - xact_start as xact_duration,
    age(backend_xmin) as xmin_age,
    *
from pg_stat_activity
where query ~ '^autovacuum'
order by 1 desc nulls last;

```

2. A long-running transaction on the primary. Use a snippet like this, in a psql session connected to primary:

```

select
    now(),
    now() - query_start as query_age,
    now() - xact_start as xact_age,
    pid,
    backend_type,
    state,
    client_addr,
    wait_event_type,
    wait_event,
    xact_start,
    query_start,
    state_change,
    query
from pg_stat_activity
where
    state != 'idle'
    and backend_type != 'autovacuum worker'
    and xact_start < now() - '60 seconds'::interval
order by xact_age desc nulls last;

```

Since we have very low values of `statement_timeout` (15s for application sessions; but 0 for `gitlab-superuser`) and `idle_in_transaction_session_timeout` (30s), in most cases (not all though!) such transaction will consist of many brief queries with brief pauses between them – so you may want to collect sampled data to understand the transaction behavior – in this case, use a shell snippet like this one, collecting samples to a CSV file (replace XXX in the filename with issue number like `production-1234`):

```

while sleep 1; do
    sudo gitlab-psql -taX -c "
        copy(select
            now(),
            now() - query_start as query_age,
            now() - xact_start as xact_age,
            pid,
            backend_type,

```

```

        state,
        client_addr,
        wait_event_type,
        wait_event,
        xact_start,
        query_start,
        state_change,
        query
    from pg_stat_activity
    where
        state != 'idle'
        and backend_type != 'autovacuum worker'
        and xact_start < now() - '60 seconds'::interval
    order by xact_age desc nulls last) to stdout csv" \
    | tee -a issue_XXX_long_tx_sampling.csv
done

```

Note that if long-running transaction has already finished and it had queries lasting less than `log_min_duration_statement` (currently 1s), the PostgreSQL log won't have any entries logged for such a transaction, so we won't be able to understand what caused this transaction and what queries it had. That's why it is important to collect samples when it's still happening.

3. A long-running transaction on a standby with `hot_standby_feedback = on`. To identify such standby, check `pg_replication_slots` on the primary – the standby causing the issue will have lagging `xmin` value (use `age(xmin)`). Then connect to that standby and use the query above to identify long-running transactions.

Other interesting related metrics that are available:

- Long running transactions in prometheus
- Checking Logs with `Gitlab::Database::Transaction::Context` class

Mitigation

Once the root cause of the `pg_txid_xmin_age` spike is identified, consider interrupting the offending session using `pg_terminate_backend({PID})`:

- for `autoANALYZE`, it won't cause issue (but recommended to run a manual `analyze (verbose, skip_locked) {table_name}` right after interrupting, ensuring that `autoANALYZE` didn't start to compete with our manual attempt);
- for a regular long-running transactions – it is a business decision. Consequences of transaction being interrupted need to be analyzed. Interruption can be done using `pg_terminate_backend({PID})` as well. In any case,

consider opening a <https://gitlab.com/gitlab-org/gitlab> issue so that product development teams can look into potential underlying problems.

```
select pg_terminate_backend(1451058);
```

Post-checks

Once the root cause of the `pg_txid_xmin_age` spike is eliminated, ensure that:

- the spike has ended,
- the 5xx errors, slow Postgres queries have ended.